Experiment No. 6

Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE

Aim

To create and deploy the Ballot (voting) smart contract using Solidity on Remix IDE, changing the proposal name type from `bytes32` to `string`, and to perform transactions such as giving the right to vote, voting and delegation.

Course Outcomes: CO3, CO5, CO6

Theory

The **Ballot** contract implements a voting process with vote delegation. The chairperson gives voters the right to vote; each voter can vote for a proposal or delegate the vote to another voter, and the proposal with the most votes wins.

Relevance of `require` – a guard condition. If the condition is false, execution stops and all state changes of the transaction are reverted, with an optional error message. In Ballot it checks that only the chairperson gives voting rights, that a voter has not already voted, and that the caller has the right to vote.

mapping, storage and memory

- `mapping(address => Voter)` – hash-table-like structure linking each address to its voter data; cannot be iterated.
- `storage` – permanent data on the blockchain (state variables); costs more gas.
- `memory` – temporary data that exists only during a function call; cheaper.

Why `bytes32` instead of `string`? `bytes32` is a fixed-size type, so it is cheaper in gas and faster to compare, but limited to 32 bytes and unreadable. `string` is dynamically sized, readable and easy to use, but costs more gas.

Requirements

- Computer with a web browser and internet connection.
- Remix IDE (https://remix.ethereum.org) – Remix VM (Osaka) environment.
- Solidity compiler 0.8.34 and the `Ballot.sol` contract.

Procedure

Step 1: Open the contract
Open Remix IDE, go to Workspaces → `contracts` and open `3_Ballot.sol`.

Step 2: Change bytes32 to string
In `struct Proposal` use `string name;`, in the constructor use `string[] memory proposalNames`, and in `winnerName()` return `string memory winnerName_`.

Step 3: Compile
Open the Solidity Compiler tab and compile `3_Ballot.sol`.

Step 4: Deploy
In Deploy & Run, select Remix VM and Account 1 (chairperson). Enter `["Alice","Bob","Charlie"]` as proposalNames and click Deploy.

Step 5: Give the right to vote
From Account 1, call `giveRightToVote` with the addresses of Accounts 2, 3 and 4. Verify with `voters`.

Step 6: Vote
Switch to Account 2 and call `vote(1)`.

Step 7: Delegate
Switch to Account 4 and call `delegate` with the address of Account 3. Verify the weight of Account 3 with `voters`.

Accounts used:

Account	Role	Address
Account 1	Chairperson	0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
Account 2	Voter (votes)	0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2
Account 3	Voter (delegate)	0x4B20993Bc481177ec7E8f571ceCaE8A9e22C02db
Account 4	Voter (delegator)	0x78731D3Ca6b7E34aC0F824c42a7cC18A495cabaB

Code

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {
    // This declares a new complex type which will
    // be used for variables later.
    // It will represent a single voter.
    struct Voter {
        uint weight; // weight is accumulated by delegation
        bool voted; // if true, that person already voted
        address delegate; // person delegated to
        uint vote; // index of the voted proposal
    }

    // This is a type for a single proposal.
    struct Proposal {
        // If you can limit the length to a certain number of bytes,
        // always use one of bytes1 to bytes32 because they are much cheaper
        string name; // short name (up to 32 bytes)
        uint voteCount; // number of accumulated votes
    }

    address public chairperson;

    // This declares a state variable that
    // stores a 'Voter' struct for each possible address.
    mapping(address => Voter) public voters;

    // A dynamically-sized array of 'Proposal' structs.
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to choose one of 'proposalNames'.
     * @param proposalNames names of proposals
     */
    constructor(string[] memory proposalNames) {
```

```
        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        // For each of the provided proposal names,
        // create a new proposal object and add it
        // to the end of the array.
        for (uint i = 0; i < proposalNames.length; i++) {
            // 'Proposal({...})' creates a temporary
            // Proposal object and 'proposals.push(...)'
            // appends it to the end of 'proposals'.
            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }

     /**
     * @dev Give 'voter' the right to vote on this ballot. May only be called by 'chairperson'.
     * @param voter address of voter
     */
    function giveRightToVote(address voter) external {
        // If the first argument of `require` evaluates
        // to 'false', execution terminates and all
        // changes to the state and to Ether balances
        // are reverted.
        // This used to consume all gas in old EVM versions, but
        // not anymore.
        // It is often a good idea to use 'require' to check if
        // functions are called correctly.
        // As a second argument, you can also provide an
        // explanation about what went wrong.
        require(
            msg.sender == chairperson,
            "Only chairperson can give right to vote."
        );
        require(
            !voters[voter].voted,
            "The voter already voted."
        );
        require(voters[voter].weight == 0, "Voter already has the right to vote.");
        voters[voter].weight = 1;
    }

    /**
     * @dev Delegate your vote to the voter 'to'.
     * @param to address to which vote is delegated
     */
    function delegate(address to) external {
        // assigns reference
        Voter storage sender = voters[msg.sender];
        require(sender.weight != 0, "You have no right to vote");
        require(!sender.voted, "You already voted.");

        require(to != msg.sender, "Self-delegation is disallowed.");

        // Forward the delegation as long as
        // 'to' also delegated.
        // In general, such loops are very dangerous,
        // because if they run too long, they might
        // need more gas than is available in a block.
        // In this case, the delegation will not be executed,
        // but in other situations, such loops might
        // cause a contract to get "stuck" completely.
        while (voters[to].delegate != address(0)) {
            to = voters[to].delegate;

            // We found a loop in the delegation, not allowed.
            require(to != msg.sender, "Found loop in delegation.");
        }

        Voter storage delegate_ = voters[to];
```

```
        // Voters cannot delegate to accounts that cannot vote.
        require(delegate_.weight >= 1);

        // Since 'sender' is a reference, this
        // modifies 'voters[msg.sender]'.
        sender.voted = true;
        sender.delegate = to;

        if (delegate_.voted) {
            // If the delegate already voted,
            // directly add to the number of votes
            proposals[delegate_.vote].voteCount += sender.weight;
        } else {
            // If the delegate did not vote yet,
            // add to her weight.
            delegate_.weight += sender.weight;
        }
    }

    /**
     * @dev Give your vote (including votes delegated to you) to proposal 'proposals[proposal].name'.
     * @param proposal index of proposal in the proposals array
     */
    function vote(uint proposal) external {
        Voter storage sender = voters[msg.sender];
        require(sender.weight != 0, "Has no right to vote");
        require(!sender.voted, "Already voted.");
        sender.voted = true;
        sender.vote = proposal;

        // If 'proposal' is out of the range of the array,
        // this will throw automatically and revert all
        // changes.
        proposals[proposal].voteCount += sender.weight;
    }

    /**
     * @dev Computes the winning proposal taking all previous votes into account.
     * @return winningProposal_ index of winning proposal in the proposals array
     */
    function winningProposal() public view
            returns (uint winningProposal_)
    {
        uint winningVoteCount = 0;
        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }

    /**
     * @dev Calls winningProposal() function to get the index of the winner contained in the proposals
          array and then
     * @return winnerName_ the name of the winner
     */
    function winnerName() external view
            returns (string memory winnerName_)
    {
        winnerName_ = proposals[winningProposal()].name;
    }
}
```

Output

Compile

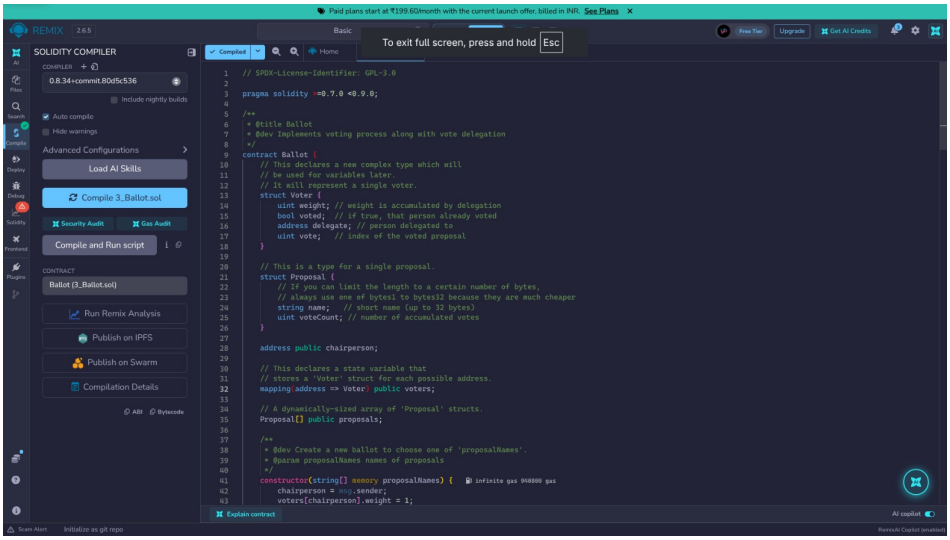


Figure 1: Ballot.sol compiled successfully (Solidity 0.8.34)

Deploy

The contract was deployed from Account 1 with the proposal names as strings. `proposals(2)` returns the name `Charlie` with `voteCount 0`.

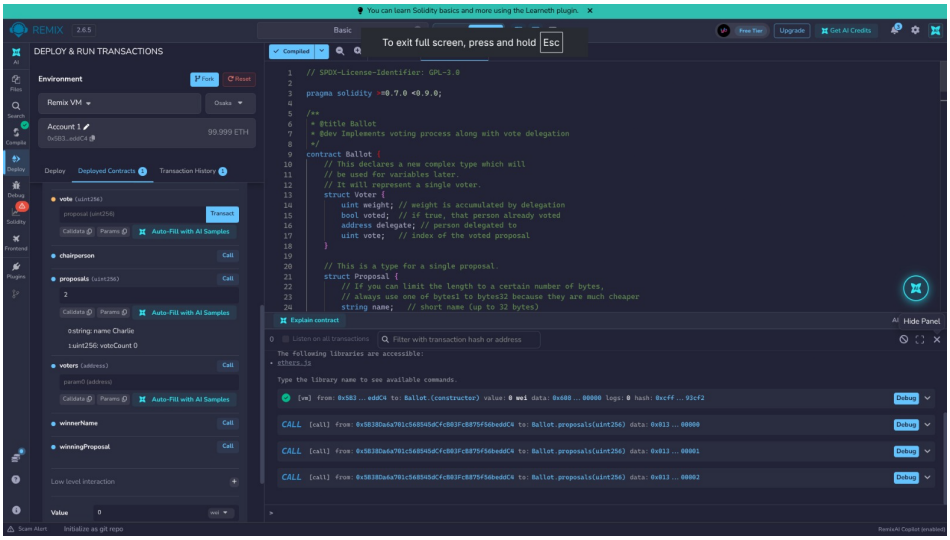


Figure 2: Ballot deployed on Remix VM; proposals(2) returns Charlie

Give the right to vote
Account 1 gave the right to vote to Account 2. `voters` shows weight 1 and voted false.

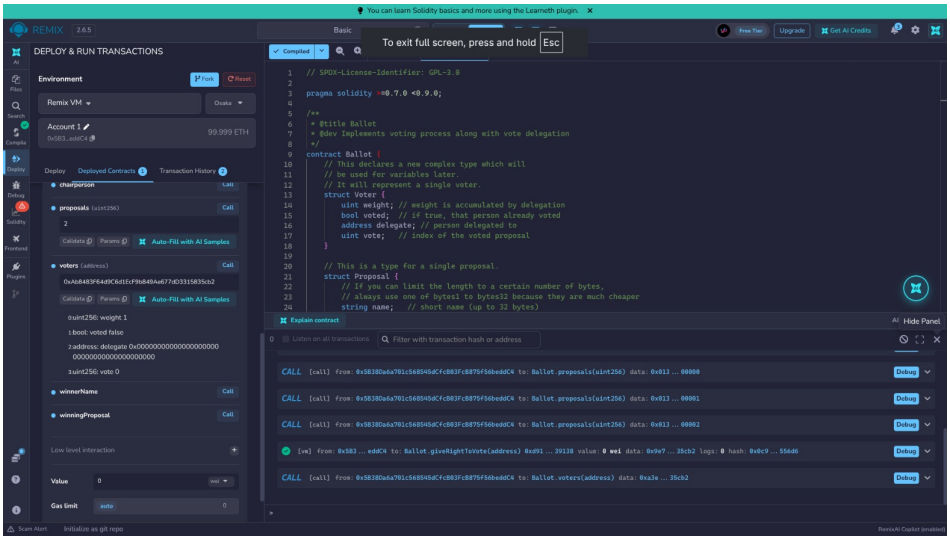


Figure 3: giveRightToVote by chairperson and voters(Account 2)

Vote
Account 2 voted for proposal 1. `proposals(1)` shows `Bob` with `voteCount 1`.

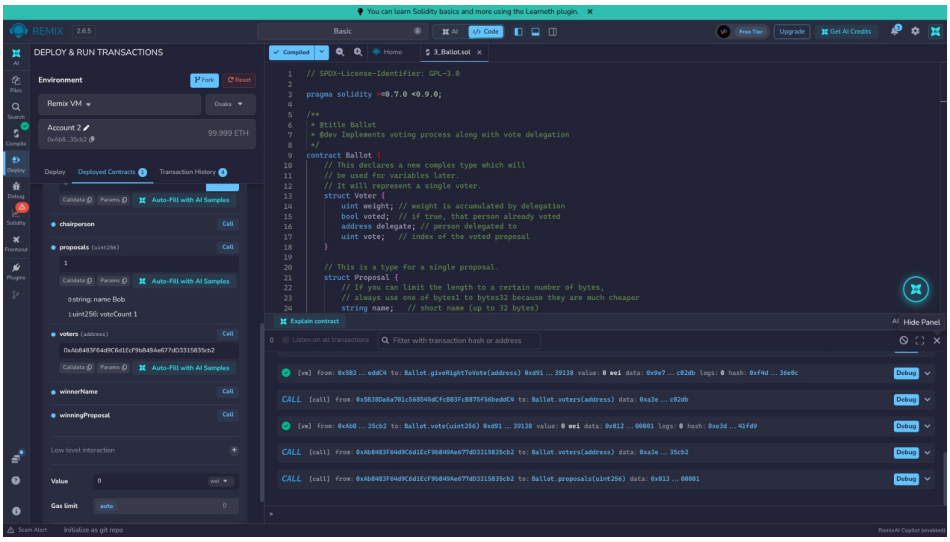


Figure 4: vote(1) by Account 2 and proposals(1)

Delegate
Account 4 delegated its vote to Account 3. The weight of Account 3 became 2 (own vote plus the delegated vote).

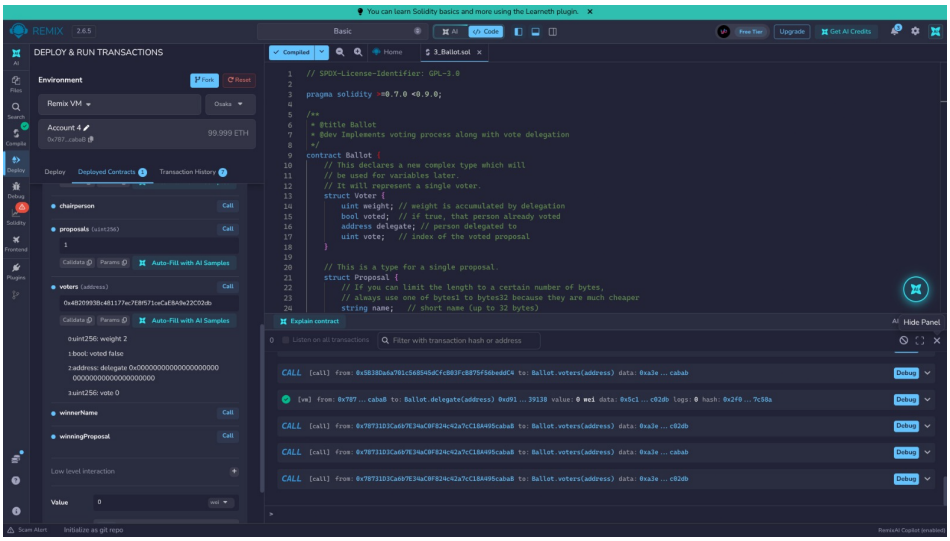


Figure 5: delegate by Account 4; voters(Account 3) shows weight 2

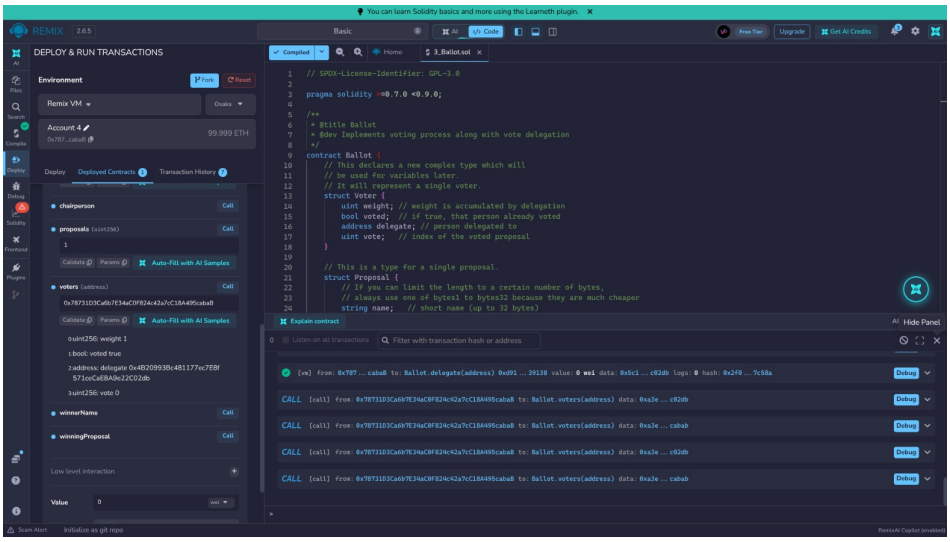


Figure 6: voters(Account 4) shows voted true and delegate = Account 3

An earlier delegate attempt by Account 4 reverted with *"You have no right to vote"*, because the chairperson had not yet given it the right to vote. This shows the `require` check at work.

Conclusion

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.

Experiment completed successfully.